

Unit-3: Database Interaction and CodeIgniter Framework

3.1 PHP with MySQL/MongoDB

3.1.1 Connecting to Databases using mysqli or PDO

- PHP connects to databases like **MySQL** or **MongoDB** to store and manage data.
- For MySQL, PHP offers two main ways:
 1. **MySQLi** – "i" means **improved**. Procedural or Object-Oriented.
 2. **PDO (PHP Data Object)** – works with **many databases** like MySQL, SQLite, etc.

✦ Need Before Connecting:

- XAMPP should be running (Apache + MySQL)
- A database (e.g., mydb) already created in **phpMyAdmin**
- A table

◆ Method 1: MySQLi (Procedural Style)

```
<?php

$host = "localhost";

$username = "root";

$password = "";

$dbname = "mydb";


// Connect to database

$conn = mysqli_connect($host, $username, $password, $dbname);

// Check connection

if (!$conn) {

    die("Connection failed: " . mysqli_connect_error());

}

echo "Connected successfully!";

?>
```

Unit-3: Database Interaction and CodeIgniter Framework

Output:

Connected successfully!

◆ Method 2: PDO (Object-Oriented)

```
<?php

$host = "localhost";

$dbname = "mydb";

$username = "root";

$password = "";

try {

    // Create PDO connection

    $conn = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);

    // Set error mode

    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    echo "Connected successfully (PDO)";

} catch (PDOException $e) {

    echo "Connection failed: " . $e->getMessage();

}

?>
```

3.1.2 Creating Databases and Tables using PHP

- A **database** stores all your data (like student records, products, users, etc.).
- A **table** is like an Excel sheet inside the database.
- We use SQL commands inside PHP to **create a database or table**.

1. Creating a Database using PHP + MySQLi

```
<?php
```

Unit-3: Database Interaction and CodeIgniter Framework

```
$host = "localhost";

$username = "root";

$password = "";


// Step 1: Connect to MySQL server (no database yet)

$conn = mysqli_connect($host, $username, $password);

// Step 2: Check connection

if (!$conn) {

    die("Connection failed: " . mysqli_connect_error());

}

// Step 3: Create database

$sql = "CREATE DATABASE mydb";

if (mysqli_query($conn, $sql)) {

    echo "Database 'mydb' created successfully!";

} else {

    echo "Error: " . mysqli_error($conn);

}

mysqli_close($conn); // Step 4: Close connection

?>
```

Output:

Database 'mydb' created successfully!

2. Creating a Table inside the Database

```
<?php

$host = "localhost";

$username = "root";

$password = "";
```

Unit-3: Database Interaction and CodeIgniter Framework

```
$database = "mydb";
```

```
// Step 1: Connect to database
```

```
$conn = mysqli_connect($host, $username, $password, $database);
```

```
// Step 2: Check connection
```

```
if (!$conn) {
```

```
    die("Connection failed: " . mysqli_connect_error());
```

```
}
```

```
// Step 3: Create table (students)
```

```
$sql = "CREATE TABLE students (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    name VARCHAR(100),
```

```
    email VARCHAR(100),
```

```
    age INT
```

```
);
```

```
if (mysqli_query($conn, $sql)) {
```

```
    echo "Table 'students' created successfully!";
```

```
} else {
```

```
    echo "Error: " . mysqli_error($conn);
```

```
}
```

```
mysqli_close($conn); // Step 4: Close connection
```

```
?>
```

Output:

```
Table 'students' created successfully!
```

Unit-3: Database Interaction and CodeIgniter Framework

3.1.3 CRUD Operations: INSERT, SELECT, UPDATE, DELETE

- **CRUD** = Basic operations to manage data in a database table.

Operation	Meaning	SQL Command
C	Create new record	INSERT
R	Read data	SELECT
U	Update existing data	UPDATE
D	Delete data	DELETE

Table: students

Columns: id, name, email, age

1. INSERT (Create)

```
<?php
```

```
$conn = mysqli_connect("localhost", "root", "", "mydb");
```

```
$sql = "INSERT INTO students (name, email, age)
```

```
VALUES ('Neha', 'Neha@gmail.com', 22)";
```

```
if (mysqli_query($conn, $sql)) {
```

```
    echo "New record inserted!";
```

```
} else {
```

```
    echo "Error: " . mysqli_error($conn);
```

```
}
```

```
?>
```

Output:

New record inserted!

Unit-3: Database Interaction and CodeIgniter Framework

2. SELECT (Read)

```
<?php

$conn = mysqli_connect("localhost", "root", "", "mydb");

$sql = "SELECT * FROM students";

$result = mysqli_query($conn, $sql);

while ($row = mysqli_fetch_assoc($result)) {

    echo "ID: " . $row["id"] . " | Name: " . $row["name"] .

        " | Email: " . $row["email"] . " | Age: " . $row["age"] . "<br>";

}

?>
```

Output:

ID: 1 | Name: Neha | Email: Neha@gmail.com | Age: 22

3. UPDATE (Change existing record)

```
<?php

$conn = mysqli_connect("localhost", "root", "", "mydb");

$sql = "UPDATE students SET age = 23 WHERE id = 1";

if (mysqli_query($conn, $sql)) {

    echo "Record updated!";

} else {

    echo "Error: " . mysqli_error($conn);

}

?>
```

Output:

Record updated!

Unit-3: Database Interaction and CodeIgniter Framework

4. DELETE (Remove a record)

```
<?php

$conn = mysqli_connect("localhost", "root", "", "mydb");

$sql = "DELETE FROM students WHERE id = 1";

if (mysqli_query($conn, $sql)) {

    echo "Record deleted!";

} else {

    echo "Error: " . mysqli_error($conn);

}

?>
```

Output:

Record deleted!

Example: Insert and Display Student Records using HTML + PHP + MySQLi

1. form.html – User Input Form

```
<!-- form.html -->

<h2>Student Entry Form</h2>

<form action="insert.php" method="post">

    <table border="1" cellpadding="5">

        <tr>

            <td>Name:</td>

            <td><input type="text" name="name"></td>

        </tr>

        <tr>

            <td>Email:</td>

            <td><input type="text" name="email"></td>

        </tr>

    </table>

</form>
```

Unit-3: Database Interaction and CodeIgniter Framework

```
</tr>

<tr>

    <td>Age:</td>

    <td><input type="number" name="age"></td>

</tr>

<tr>

    <td colspan="2"><input type="submit" value="Insert Record"></td>

</tr>

</table>

</form>
```

2. insert.php – Save Data to Database

```
<?php

// Database connection

$conn = mysqli_connect("localhost", "root", "", "mydb");

$name = $_POST['name'];

$email = $_POST['email'];

$age = $_POST['age'];

// SQL to insert

$sql = "INSERT INTO students (name, email, age)

        VALUES ('$name', '$email', $age)";

if (mysqli_query($conn, $sql)) {

    echo "Record Inserted Successfully!<br><br>";

} else {

    echo "Error: " . mysqli_error($conn);

}
```


Unit-3: Database Interaction and CodeIgniter Framework

```
// Show data after insert
```

```
include("display.php");
```

```
?>
```

3. display.php – Show Records in Table

```
<?php
```

```
$conn = mysqli_connect("localhost", "root", "", "mydb");
```

```
$sql = "SELECT * FROM students";
```

```
$result = mysqli_query($conn, $sql);
```

```
echo "<h2>Student Records</h2>";
```

```
echo "<table border='1' cellpadding='5'>
```

```
    <tr><th>ID</th><th>Name</th><th>Email</th><th>Age</th></tr>";
```

```
while ($row = mysqli_fetch_assoc($result)) {
```

```
    echo "<tr>
```

```
        <td>{$row['id']}</td>
```

```
        <td>{$row['name']}</td>
```

```
        <td>{$row['email']}</td>
```

```
        <td>{$row['age']}</td>
```

```
    </tr>";
```

```
}
```

```
echo "</table>";
```

```
?>
```

★ Student Management with Update & Delete

2. Update display.php to show buttons

```
<?php
```

```
$conn = mysqli_connect("localhost", "root", "", "mydb");
```

Unit-3: Database Interaction and CodeIgniter Framework

```
$sql = "SELECT * FROM students";

$result = mysqli_query($conn, $sql);

echo "<h2>Student Records</h2>";

echo "<table border='1' cellpadding='5'>

<tr>

    <th>ID</th><th>Name</th><th>Email</th><th>Age</th><th>Actions</th>

</tr>";

while ($row = mysqli_fetch_assoc($result)) {

    echo "<tr>

        <td>{$row['id']}</td>

        <td>{$row['name']}</td>

        <td>{$row['email']}</td>

        <td>{$row['age']}</td>

        <td>

            <a href='edit.php?id={$row['id']}'>Edit</a> |

            <a href='delete.php?id={$row['id']}' onclick='return confirm(\"Are you sure?\")>Delete</a>

        </td>

    </tr>";

}

echo "</table>";

?>
```

3. delete.php – Delete record by ID

```
<?php

$conn = mysqli_connect("localhost", "root", "", "mydb");

$id = $_GET['id']; // Get ID from URL

$sql = "DELETE FROM students WHERE id = $id";
```

Unit-3: Database Interaction and CodeIgniter Framework

```
if (mysqli_query($conn, $sql)) {  
    echo "Record deleted!";  
} else {  
    echo "Error deleting record: " . mysqli_error($conn);  
}  
  
// Redirect back  
  
header("Location: form.html");  
  
?>
```

4. edit.php – Show editable form for update

```
<?php  
  
$conn = mysqli_connect("localhost", "root", "", "mydb");  
  
$id = $_GET['id']; // Get ID to edit  
  
$sql = "SELECT * FROM students WHERE id = $id";  
  
$result = mysqli_query($conn, $sql);  
  
$row = mysqli_fetch_assoc($result);  
  
?>  
  
<h2>Edit Student</h2>  
  
<form action="update.php" method="post">  
  
    <input type="hidden" name="id" value="<?= $row['id'] ?>">  
  
    <table border="1" cellpadding="5">  
  
        <tr><td>Name:</td><td><input type="text" name="name" value="<?= $row['name']  
?>"></td></tr>  
  
        <tr><td>Email:</td><td><input type="text" name="email" value="<?= $row['email']  
?>"></td></tr>  
  
        <tr><td>Age:</td><td><input type="number" name="age" value="<?= $row['age']
```

Unit-3: Database Interaction and CodeIgniter Framework

```
?>"</td></tr>

<tr><td colspan="2"><input type="submit" value="Update"></td></tr>

</table>

</form>
```

5. update.php – Process update

```
<?php

$conn = mysqli_connect("localhost", "root", "", "mydb");

$id = $_POST['id'];
$name = $_POST['name'];
$email = $_POST['email'];
$age = $_POST['age'];

$sql = "UPDATE students SET name='$name', email='$email', age=$age WHERE id=$id";
if (mysqli_query($conn, $sql)) {
    echo "Record updated!";
} else {
    echo "Update failed: " . mysqli_error($conn);
}

// Redirect back
header("Location: form.html");

?>
```

3.1.4 Using SQL Clauses: WHERE, ORDER BY, LIMIT

◆ What are SQL Clauses?

- SQL clauses are special keywords used in queries to **filter**, **sort**, and **control** the data output from a database table.
- These clauses help make our queries more meaningful, useful, and customized.

1. WHERE Clause — Filter Data

- WHERE is used to fetch **only those records** that match a given condition.
- It is commonly used with SELECT, UPDATE, and DELETE statements.

■ Syntax:

```
SELECT * FROM table_name WHERE condition;
```

💡 Example:

```
SELECT * FROM students WHERE age = 20;
```

2. ORDER BY Clause — Sort Data

- ORDER BY is used to sort the result in **ascending** (ASC) or **descending** (DESC) order.
- It can be applied on one or more columns.

■ Syntax:

```
SELECT * FROM table_name ORDER BY column_name ASC|DESC;
```

💡 Example:

```
SELECT * FROM students ORDER BY age ASC;
```

3. LIMIT Clause — Control the Number of Records

- LIMIT is used to restrict the number of rows returned from a query.
- It is very useful when we want to fetch only a few top or latest records.

■ Syntax:

```
SELECT * FROM table_name LIMIT number;
```

💡 Example: SELECT * FROM students LIMIT 5;

Unit-3: Database Interaction and CodeIgniter Framework

3.2 AJAX for Backend Integration

3.2.1 Introduction to AJAX and Asynchronous Requests

◆ What is AJAX?

- **AJAX** stands for **Asynchronous JavaScript and XML**.
- It allows **web pages to send and receive data from the server in the background** without refreshing the page.
- **AJAX uses a combination of:**
 - HTML (for page structure)
 - JavaScript (for making requests)
 - PHP (for backend processing)
 - XML or JSON (for sending/receiving data — JSON is more common now)

★ Why use AJAX?

Traditional Web	AJAX Web
Page reloads fully after every request	Page updates without refreshing
Slower and less user-friendly	Faster and dynamic
Example: Submit a form → page reloads	Example: Search while typing (live search)

🔄 Synchronous vs. Asynchronous:

Type	Meaning
Synchronous	Tasks happen one after another; browser waits
Asynchronous	Tasks happen in background; browser does not wait

AJAX works **asynchronously**.

★ Examples of AJAX:

Application	Feature using AJAX
-------------	--------------------

Unit-3: Database Interaction and CodeIgniter Framework

Google Search	Live search suggestions
Facebook Likes/Comments	Load without refreshing
Online forms	Validation without reload
Weather apps	Auto-update temperature

✦ Components of AJAX:

Component	Role
JavaScript	Sends the request to server
XMLHttpRequest or fetch()	Makes the actual request
PHP	Server-side logic and response
JSON	Format for sending data back

3.2.2 Sending AJAX Requests to PHP

- AJAX stands for **Asynchronous JavaScript and XML**.
- It is used to send or receive data between **browser (frontend)** and **server (backend)** without **reloading the page**.
- XMLHttpRequest is a built-in JavaScript object that allows AJAX communication.
- PHP receives data using \$_GET or \$_POST, processes it, and returns a result.
- The response is displayed on the webpage without refresh.

◆ Main Steps in AJAX Request:

1. JavaScript captures input from the user.
2. XMLHttpRequest sends data to a PHP file.
3. PHP processes the data and returns a result.
4. JavaScript receives the response and updates the page.

Unit-3: Database Interaction and CodeIgniter Framework

Example: Display current date from server

➤ **Index.html**

```
<button onclick="getDate()">Show Date</button>

<p id="output"></p>

<script>

function getDate() {

    var xhttp = new XMLHttpRequest();

    xhttp.onreadystatechange = function () {

        if (this.readyState == 4 && this.status == 200) {

            document.getElementById("output").innerHTML = this.responseText;

        }

    };

    xhttp.open("GET", "date.php", true);

    xhttp.send();

}

</script>
```

➤ **Date.php**

```
<?php

echo "Today is: " . date("Y-m-d");

?>
```

3.2.3 Real-Time Search Functionality

- Real-time search means the results appear **while user is typing**, without reloading the page.
- It uses **AJAX** to send the keyword to a **PHP file**, which searches the database and sends back the result.
- This improves user experience and speed, just like search suggestions in Google or YouTube.

Unit-3: Database Interaction and CodeIgniter Framework

◆ Technologies Used:

Tool	Purpose
HTML	Input box for search
JavaScript	AJAX request to server
PHP	Fetch matching records from database
MySQL	Store sample data

3.2.4 JSON Data Exchange with JavaScript and PHP

- **JSON** stands for **JavaScript Object Notation**.
- JSON is a **lightweight, text-based** format for sending data between **JavaScript and PHP**.
- Data can be easily **converted from array** → **JSON** using `json_encode()` in PHP.
- JavaScript can convert JSON into a usable object using `JSON.parse()`.

◆ Why use JSON?

Feature	Description
Easy to use	Simple key-value format
Fast	Loads faster than XML
Language-neutral	Used with many languages
Human-readable	Easy to read and debug

3.3 CodeIgniter Introduction

3.3.1 Installing and Configuring CodeIgniter (CI4)

- **CodeIgniter** is a **PHP framework** based on **MVC (Model-View-Controller)**.
- It helps developers write **organized and clean code** faster.
- Version **CI4** is the latest and modern version (faster, secure, and lightweight).

Unit-3: Database Interaction and CodeIgniter Framework

- CodeIgniter follows **URL routing**, has built-in **form validation**, **database connection**, and **helper libraries**.

✂ Installation and Configuration Steps (for CI4):

◆ Step 1: System Requirements

Requirement	Minimum
PHP	8.1 or higher
MySQL	Installed (for database)
Server	XAMPP / WAMP / LAMP
Composer	Installed (Recommended for CI4 setup)

◆ Step 2: Download CodeIgniter 4

Method 1: Manual Download

- Visit official site: <https://codeigniter.com>
- Download **CodeIgniter 4 zip**
- Extract it in htdocs (XAMPP) or www (WAMP)

Example Path:

C:/xampp/htdocs/myproject/

Method 2: Composer (Recommended)

```
composer create-project codeigniter4/appstarter myproject
```

◆ Step 3: Folder Structure (Important Folders)

Folder	Purpose
app/	Application code (MVC files)
public/	Root directory for browser (contains index.php)
system/	Core CodeIgniter files (don't edit)
writable/	Logs, cache, uploads, etc.

Unit-3: Database Interaction and CodeIgniter Framework

◆ Step 4: Run the Project

Method A: Using php spark serve

1. Open terminal / command prompt
2. Go to project folder

```
cd C:/xampp/htdocs/myproject
```

```
php spark serve
```

3. Open browser:

```
http://localhost:8080
```

✓ Welcome page will appear.

Method B: Using Browser + XAMPP

1. Place folder inside htdocs (like myproject)
2. Start Apache from XAMPP
3. In browser, type:

```
http://localhost/myproject/public
```

✓ User will see the default CodeIgniter welcome page.

✦ Configuration: app/Config/App.php

Change base URL:

```
public string $baseUrl = 'http://localhost/myproject/public/';
```

3.3.2 Understanding MVC Architecture in CodeIgniter

- MVC stands for **Model-View-Controller**
- It is a **design pattern** used to organize code in **3 separate parts**
- CodeIgniter (CI4) follows MVC to separate:
 - **Business logic (Model)**
 - **Display/Design (View)**

Unit-3: Database Interaction and CodeIgniter Framework

- Control logic (Controller)

◆ MVC Structure:

Component	Role / Responsibility
Model	Handles data (like DB operations, queries)
View	Handles UI (HTML output shown to user)
Controller	Connects Model and View (logic + routing)

How It Works:

1. User opens browser and types: `http://localhost/myproject/public/home`
2. CodeIgniter loads **Home Controller**
3. Controller fetches data (from Model if needed)
4. Controller sends data to View
5. View displays output on screen

✓ Example – MVC in CI4

■ Folder: `app/Controllers/Home.php` → Controller

```
<?php
```

```
namespace App\Controllers;
```

```
class Home extends BaseController
```

```
{
```

```
    public function index()
```

```
    {
```

```
        // Send data to view
```

```
        $data['title'] = "Welcome to TYBCA";
```

```
        return view('welcome', $data);
```

Unit-3: Database Interaction and CodeIgniter Framework

```
}  
  
}
```

■ Folder: `app/Views/welcome.php` → View

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
  
    <title>My CI4 Page</title>  
  
</head>  
  
<body>  
  
    <h2><?= $title ?></h2>  
  
    <p>This is a view file connected using MVC.</p>  
  
</body>  
  
</html>
```

🔄 What Happens?

- When user opens:
 - `http://localhost/myproject/public/home`
 - Controller (Home) runs the `index()` method
 - Sends `$title` to `welcome.php`
 - View displays it using `<?= $title ?>`
-

3.3.3 Creating Models, Views, and Controllers

- **CodeIgniter MVC** helps in separating code for:
 - Data (Model)
 - Logic (Controller)
 - Display (View)
- This makes the project **clean, reusable, and manageable**.

Unit-3: Database Interaction and CodeIgniter Framework

◆ 3 Parts of MVC

Part	Description
Model	Handles database operations
View	Displays HTML output
Controller	Controls the flow (calls model + loads view)

✓ Example: Show Student List from Database

Step 1: Create Database and Table

✓ Open **phpMyAdmin** and run:

sql

CopyEdit

```
CREATE DATABASE mycollege;
```

```
USE mycollege;
```

```
CREATE TABLE students (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(50),  
    course VARCHAR(30)  
);
```

```
INSERT INTO students (name, course) VALUES  
( 'Neha', 'BCA'),  
( 'Raj', 'BSc'),  
( 'Krishna', 'MCA');
```

Unit-3: Database Interaction and CodeIgniter Framework

Step 2: Configure Database in CodeIgniter

■ File: app/Config/Database.php

Update the default group (lines below):

php

CopyEdit

```
public $default = [  
    'DSN' => "",  
    'hostname' => 'localhost',  
    'username' => 'root',  
    'password' => "",  
    'database' => 'mycollege',  
    'DBDriver' => 'MySQLi',  
];
```

Step 3: Create Model

■ File: app/Models/StudentModel.php

php

CopyEdit

<?php

```
namespace App\Models;
```

```
use CodeIgniter\Model;
```

```
class StudentModel extends Model
```

```
{
```

```
    protected $table = 'students';
```

Unit-3: Database Interaction and CodeIgniter Framework

```
protected $allowedFields = ['name', 'course'];  
}
```

Step 4: Create Controller

■ File: app/Controllers/Student.php

php

CopyEdit

<?php

```
namespace App\Controllers;  
  
use App\Models\StudentModel;  
  
class Student extends BaseController  
{  
  
    public function index()  
    {  
  
        $model = new StudentModel();  
  
  
        // Get all student records  
  
        $data['students'] = $model->findAll();  
  
  
        // Load view with data  
  
        return view('student_view', $data);  
    }  
}
```

Step 5: Create View

■ File: app/Views/student_view.php

Unit-3: Database Interaction and CodeIgniter Framework

php

CopyEdit

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Student List</title>
```

```
</head>
```

```
<body>
```

```
<h2>Student Records</h2>
```

```
<table border="1" cellpadding="8">
```

```
    <tr>
```

```
        <th>ID</th>
```

```
        <th>Name</th>
```

```
        <th>Course</th>
```

```
    </tr>
```

```
<?php foreach ($students as $student): ?>
```

```
<tr>
```

```
    <td><?= $student['id'] ?></td>
```

```
    <td><?= $student['name'] ?></td>
```

```
    <td><?= $student['course'] ?></td>
```

```
</tr>
```

```
<?php endforeach; ?>
```

```
</table>
```

```
</body> </html>
```

Step 6: Add Route

■ File: app/Config/Routes.php

Unit-3: Database Interaction and CodeIgniter Framework

php

CopyEdit

```
$routes->get('/students', 'Student::index');
```

Run Project

Open in browser:

ruby

CopyEdit

<http://localhost/myproject/public/students>

✓ Output:

markdown

CopyEdit

Student Records

ID | Name | Course

1 | Neha | BCA

2 | Raj | BSc

3 | Krishna | MCA

3.3.4 URL Routing and Default Controller Setup

◆ What is URL Routing?

Routing is the process of mapping **URL requests** to specific **controller classes and methods** in CodeIgniter. It defines **how CodeIgniter handles incoming requests** based on the URL segments provided by the user.

For example, if a user enters <http://localhost/myapp/public/student/profile/3>, the routing system decides **which controller (Student) and which method (profile)** should handle that request.

◆ Purpose of Routing

Routing allows developers to:

Unit-3: Database Interaction and CodeIgniter Framework

- Organize URLs logically
- Hide internal controller/method names
- Improve SEO-friendliness
- Control how the application responds to different types of requests (GET, POST, etc.)

◆ Default Controller

The **default controller** is the one that is executed **when no controller is specified in the URL**. This is usually the home or landing page of the application.

In CodeIgniter, this is defined in the app/Config/Routes.php file like this:

```
$routes->setDefaultController('Home');
```

If the user accesses the base URL of the site (e.g., <http://localhost/myapp/public/>), CodeIgniter will automatically call the `index()` method of the Home controller.

◆ Default Method

If no method is specified in the URL, CodeIgniter will use the default method, which is usually:

```
$routes->setDefaultMethod('index');
```

This means that if the URL is simply `/student`, CodeIgniter will execute `Student::index()`.

◆ Auto Routing

Auto Routing allows the framework to automatically match URL patterns to controller and method names without manually defining each one.

```
$routes->setAutoRoute(true);
```

With this setting, CodeIgniter will interpret URLs like `/student/profile/5` as:

- Controller: Student
- Method: profile
- Parameter: 5

However, enabling auto routing may expose the application to **security risks**, especially if there are sensitive methods that should not be accessible directly through the URL.

◆ Manual (Custom) Routing

Manual routing gives the developer full control over which URL maps to which controller and method. This is done by defining routes explicitly in `Routes.php`.

Unit-3: Database Interaction and CodeIgniter Framework

For example:

```
$routes->get('students', 'Student::index');
```

```
$routes->get('student/profile/(:num)', 'Student::profile/$1');
```

In this configuration:

- students maps to Student::index()
- student/profile/(:num) maps to Student::profile(\$id) where (:num) captures a numeric segment of the URL.

◆ Benefits of Routing System

Feature	Explanation
Centralized control	All URL mappings are defined in one place (Routes.php)
Clean and SEO-friendly URLs	Developers can hide file structure and use readable URLs
Flexibility	Developers can choose automatic or manual routing methods
Security	Developers can control which URLs can access which controllers

3.4 Core Features in CodeIgniter

3.4.1 Form Validation using CodeIgniter (CI4) Validation Library

◆ What is Form Validation?

Form validation is the process of **checking input values** submitted by the user via a form. It ensures that the data is correct, complete, and secure before being processed or stored in the database.

In CodeIgniter, this is done using the **Validation Library**, which provides a **clean and secure way** to define rules and apply them to form fields.

◆ Why is Form Validation Important?

Purpose	Description
Prevent errors	Stops invalid or incomplete data from being stored or processed
Security	Blocks harmful input like script injection (XSS)
User experience	Helps users fill out forms correctly by showing clear error messages
Data quality	Ensures only valid and expected data is submitted to the system

Unit-3: Database Interaction and CodeIgniter Framework

◆ CI4 Validation Library Overview

In CodeIgniter 4, the validation system is **loaded automatically**, and user can define **validation rules** either:

- Directly in the controller, or
- In a separate configuration file

◆ How Validation Works

1. **User submits form data**
2. **Validation rules** are applied to each field
3. If the rules pass:
 - The form proceeds (e.g., data is saved)
4. If the rules fail:
 - Errors are shown to the user
 - Form is shown again with error messages

◆ Common Validation Rules

Rule	Description
required	Field must not be empty
valid_email	Must be a valid email address
min_length	Minimum number of characters
max_length	Maximum number of characters
numeric	Must be a number
alpha	Must contain only letters
alpha_numeric	Only letters and numbers allowed
matches[field]	Must match another field (e.g., password confirmation)

◆ Defining Rules

Rules are defined in an array format like this:

```
$rules = [  
    'username' => 'required|min_length[5]|max_length[20]',  
    'email'    => 'required|valid_email',  
    'password' => 'required|min_length[8]',  
];
```

Then, validation is applied:

Unit-3: Database Interaction and CodeIgniter Framework

```
if (! $this->validate($rules)) {  
    // If validation fails  
    $data['validation'] = $this->validator;  
    return view('register_form', $data);  
} else {  
    // If validation passes  
    // Save data to database or continue  
}
```

◆ Displaying Validation Errors

Once the validation is done, the errors can be shown to the user:

```
<?php echo $validation->listErrors(); ?>
```

This displays all validation errors together. User can also show error for specific field:

```
<?php echo $validation->getError('email'); ?>
```

3.4.2 Session Management and Flashdata in CodeIgniter 4

◆ What is Session?

A **session** is a way to **store user data temporarily** across multiple pages during a visit to the website. It helps to **identify the user** and **maintain their state** (like login status, preferences, etc.).

In CodeIgniter 4, session management is **easy and secure** using its built-in **Session Library**.

◆ Types of Data Stored in Session

Type	Example
User Information	Username, User ID
Temporary Status	Login state, cart items
Flash Messages	Success or error notifications

◆ Key Features of Session in CI4

- Stores data on the **server**
- Supports different **drivers** (file, database, Redis, etc.)
- Automatically starts with each request
- Easy methods to set, get, or delete session data

Unit-3: Database Interaction and CodeIgniter Framework

◆ Common Session Methods

Method	Use
<code>\$session->set()</code>	Store data in session
<code>\$session->get()</code>	Retrieve data from session
<code>\$session->remove()</code>	Remove specific session item
<code>\$session->destroy()</code>	Clear all session data
<code>\$session->has('key')</code>	Check if session key exists

◆ What is Flashdata?

Flashdata is a type of session data that is **available only for the next request**, and then it is automatically cleared.

◆ Used mostly for **temporary messages** like:

- Success message after form submission
- Error message for invalid login

◆ Flashdata Features

Feature	Description
Temporary	Stored for one request only
Auto-delete	Deleted automatically after displaying
Useful for	Form success, login messages, error notifications

◆ Flashdata Methods

Method	Purpose
<code>\$session->setFlashdata('key', val)</code>	Set a temporary flash message
<code>\$session->getFlashdata('key')</code>	Get and display the flash message

◆ Example Flow

1. **User submits form**
2. Controller sets flashdata:
 - `$session->setFlashdata('success', 'Data saved successfully!');`
3. Redirects to next page
4. On that page, message is shown:
 - `<?php echo session()->getFlashdata('success'); ?>`

Unit-3: Database Interaction and CodeIgniter Framework

Session	Flashdata
It stays until the user is on the website or browser is closed.	It stays only for the next page or next action.
Login, cart, user details.	Messages like: "Saved successfully", "Login failed".
It works on many pages until we remove it.	It works only one time , then it is deleted.
When a user logs in, we save their ID in session.	After saving a form, we show a success message using flashdata.

3.4.3 Handling File Uploads in CodeIgniter 4

◆ What is File Upload?

File upload means sending a file (like an image, PDF, document) from the user's computer to the web server using a form. CodeIgniter 4 provides a **built-in feature** to handle file uploads easily and securely.

◆ Why File Upload is Needed?

- Uploading profile pictures
- Submitting documents (PDF, DOC, etc.)
- Uploading images in a gallery
- Uploading CV/resume in a form

◆ CI4 Uploading Process – Step-by-Step

Step	Action	Description
1	Create an HTML form	Use enctype="multipart/form-data" to allow file upload
2	Get file in Controller using request	request()->getFile('file_input_name')
3	Validate file	Check file size, type, etc. using validation rules
4	Move file to folder	Use \$file->move('upload_folder/')
5	Save file name in database	Store path or name for future reference

◆ Important Classes and Functions in CI4

Function / Class	Use
request()->getFile()	To get the uploaded file object
\$file->isValid()	To check if the file is valid
\$file->getName()	To get the name of the uploaded file
\$file->move()	To move file from temp folder to user folder

Unit-3: Database Interaction and CodeIgniter Framework

◆ Validation Rules for File Upload

Rule	Purpose
uploaded[file]	Ensure file is selected
max_size[file,1024]	Maximum file size (in KB)
mime_in[file,image/png,image/jpg]	Check file type (e.g., PNG, JPG)
ext_in[file,jpg,jpeg,png]	Allowed extensions

3.4.4 Loading Helpers and Libraries in CodeIgniter 4

◆ What are Helpers in CodeIgniter?

Helpers are simple **PHP functions** used to perform common tasks like:

- Working with URLs
- Creating form elements
- Working with arrays, strings, files, etc.

Helpers **do not use classes**. They are just **collections of functions**.

◆ Commonly Used Helpers

Helper Name	Purpose
url_helper	Create links, base URLs, site URLs
form_helper	Create form elements
text_helper	Work with text (word limit, etc.)
file_helper	File operations (read, write)
array_helper	Array manipulation

◆ Loading Helpers

User can load helpers in two ways:

1. Manually in Controller

```
helper('form'); // for form-related functions
```

```
helper(['url', 'text']); // for multiple helpers
```

2. Auto-load (Always Loaded)

```
app/Config/Autoload.php
```

Example: `public $helpers = ['url', 'form'];`

Unit-3: Database Interaction and CodeIgniter Framework

◆ What are Libraries in CodeIgniter?

Libraries are **PHP classes** that provide advanced features like:

- Session management
- File uploading
- Email sending
- Form validation

◆ Common Libraries in CI4

Library Name	Purpose
Validation	For form and data validation
Session	Store user session data
Email	Send emails
FileUpload	Manage file upload functionality
Pagination	Break data into pages

◆ Loading Libraries

In CI4, most libraries are **auto-loaded using Services**.

To use them:

```
$validation = \Config\Services::validation();
```

```
$session = \Config\Services::session();
```

Or in controller:

```
$this->session = \Config\Services::session();
```

◆ Helper vs Library

Feature	Helper	Library
Structure	Set of functions	PHP class with methods
Usage	Simple tasks	Complex operations
Example	form_open()	\$this->session->set()
Loading	helper('name')	\Config\Services::name()

Course- 504: UNIT-4 (Based on IKS)

Coruse-504 : Unit-4 (IKS based) . NOTES by : Dr. Snehal K Joshi (snehaljoshi2020@gmail.com)

(i) Presentation Topics: Students can be given following topics to prepare short presentation, focused topics ideal for 5–10 minute presentations with visual/graphical content:

1. Introduction to Suryasiddhanta and Its Historical Significance
2. Aryabhata's Concept of Earth's Rotation (Verse 3.9)
3. Determining the Length of the Solar Year (Verse 3.10)
4. Ancient Indian Understanding of Lunar Eclipses
5. Ancient Indian Understanding of Solar Eclipses
6. Aryabhata's Model of Planetary Motion (Verse 1.13)
7. Role of the Sun in Influencing Planetary Motion (Verse 2.12)
8. Zodiac Signs in Suryasiddhanta (Verse 1.5)
9. Structure of the Solar System According to Aryabhata
10. Speed of Planets in Ancient Indian Texts (Verse 6.5)
11. Distance from Earth to Moon in Suryasiddhanta (Verse 7.8)
12. Latitude and Longitude of Planets (Verse 8.12)
13. Introduction to Varahamihira and His Contributions to Astronomy
14. Lunar Phases in Ancient Indian Astronomy (Verse 2.10)
15. Concept of Sidereal and Tropical Year
16. Planetary Conjunctions: Meaning and Observations
17. Varahamihira's Observation of Star Movements (Verse 2.18)
18. Nakshatras: Fixed Stars in Indian Astronomy
19. Ecliptic Path of the Sun and Zodiac
20. Indian Methods of Time Calculation in Ancient Astronomy
21. Eclipses: Causes and Predictive Methods in Classical Texts
22. Ancient Indian Instruments Used for Sky Observation
23. Comparative Study: Aryabhata's vs. Western Views on Planetary Motion
24. The Role of Astrology in Ancient Indian Astronomy
25. Mythology and Astronomy: Cultural Significance of Celestial Events

Coruse-504 : Unit-4 (IKS based) . NOTES by : Dr. Snehal K Joshi (snehaljoshi2020@gmail.com)

(ii) Report Topics : Students can be given following topics to collect information through various reference books, websites, materials and prepare 2–4 pages reports or term papers:

1. A Comprehensive Study of Suryasiddhanta: Origin, Structure, and Influence
2. Scientific Analysis of Earth's Motion in Aryabhata's Astronomy
3. Calculating the Length of the Year in Ancient Indian Methods
4. Mechanism of Solar and Lunar Eclipses in Suryasiddhanta
5. Planetary Motions and Epicycles in Indian Astronomy
6. The Role of the Sun in Celestial Mechanics in Ancient Indian Thought
7. Zodiacal System in Indian Astronomy: Signs, Divisions, and Calculations
8. The Ancient Indian Model of the Solar System: A Scientific Review
9. Speeds of Planets and Their Implications in Jyotisha
10. Measurement of Planetary Distances: Moon, Sun, and Beyond
11. Techniques to Determine Planetary Latitude and Longitude
12. Varahamihira's Astronomical Observations and Their Scientific Value
13. Lunar Phases: Scientific Explanation and Cultural Impact
14. Stellar Movements According to Varahamihira: Ancient vs. Modern Views
15. Ecliptic Coordinates: Indian Methods for Calculating Celestial Positions
16. Understanding the Difference Between Sidereal and Tropical Years
17. Planetary Conjunctions and Their Importance in Indian Astronomy
18. Comparative Analysis of Aryabhata and Varahamihira's Astronomy
19. Astronomical Predictive Tools and Their Mathematical Foundations
20. Ancient Indian Theories of Eclipse Prediction vs. Modern Science
21. The Contribution of Indian Astronomy to World Science
22. Indian Astronomical Literature: Suryasiddhanta, Panchasiddhantika, and Others
23. Role of Mathematics in Classical Indian Astronomy
24. Transmission of Indian Astronomical Knowledge to Other Cultures
25. Observational Astronomy and Calendrical Systems in Ancient India

M.C.Q.

1. What is Aryabhata's key proposition about the motion of the Earth in Suryasiddhanta?

- ☐ A) The Earth is stationary
- ☐ B) The Earth orbits the Sun
- ☐ C) The Earth rotates on its axis
- ☐ D) The Sun revolves around Earth
- ☒ **Correct Answer: C**

2. According to Aryabhata, the apparent westward motion of stars is due to:

- ☐ A) Gravitational pull
- ☐ B) Rotation of the sky
- ☐ C) Rotation of the Earth
- ☐ D) Magnetic influence
- ☒ **Correct Answer: C**

3. Aryabhata compared the motion of stars to which of the following analogies?

- ☐ A) Birds flying in the sky
- ☐ B) Clouds floating across
- ☐ C) Boat moving in a river
- ☐ D) Trees appearing to move when in a boat
- ☒ **Correct Answer: D**

4. The concept that the **Earth rotates while the sky remains stationary** reflects:

- ☐ A) Geocentrism
- ☐ B) Heliocentrism
- ☐ C) Relative motion
- ☐ D) Absolute motion
- ☒ **Correct Answer: C**

5. Aryabhata's Earth-rotation model was advanced for its time because:

- ☐ A) It denied planetary motion
- ☐ B) It introduced gravity
- ☐ C) It explained diurnal motion accurately
- ☐ D) It matched Greek models
- ☒ **Correct Answer: C**

6. According to Aryabhata, the length of the sidereal year is approximately:
- ☐ A) 365.25 days
 - ☐ B) 365.25636 days
 - ☐ C) 360.5 days
 - ☐ D) 366 days
 - ☒ **Correct Answer: B**
7. The difference between tropical and sidereal year in Aryabhata's system is due to:
- ☐ A) Precession of the equinoxes
 - ☐ B) Elliptical orbits
 - ☐ C) Solar radiation
 - ☐ D) Magnetic fields
 - ☒ **Correct Answer: A**
8. Which unit of time did Aryabhata use for astronomical calculations?
- ☐ A) Hours and minutes
 - ☐ B) Tithis and pakshas
 - ☐ C) Kalas and vinadis
 - ☐ D) Days and months only
 - ☒ **Correct Answer: C**
9. The precision in calculating the length of a year by Aryabhata differed from the modern value by:
- ☐ A) 0.5 days
 - ☐ B) Less than 3 minutes
 - ☐ C) More than 1 day
 - ☐ D) Over 5 hours
 - ☒ **Correct Answer: B**
10. Aryabhata's calculation of the year was based on:
- A) Moon phases
 - B) Zodiac transitions
 - C) Sun's return to the same fixed star
 - D) Planetary conjunctions
 - ☒ **Correct Answer: C**
11. Aryabhata explained eclipses using which scientific principle?
- A) Demonic intervention
 - B) Shadow theory

- C) Gravitational lensing
- D) Refraction
- ☒ **Correct Answer: B**

12. According to Suryasiddhanta, a solar eclipse occurs when:

- A) Earth comes between Sun and Moon
- B) Moon comes between Sun and Earth
- C) Sun and Earth are aligned
- D) Stars align with planets
- ☒ **Correct Answer: B**

13. In lunar eclipses, Aryabhata correctly identified the shadow of:

- A) The Moon
- B) The Earth
- C) Rahu
- D) The Sun
- ☒ **Correct Answer: B**

14. Aryabhata rejected mythological explanations of eclipses in favor of:

- A) Religious beliefs
- B) Astronomical models
- C) Observations from priests
- D) Zodiac predictions
- ☒ **Correct Answer: B**

15. The Suryasiddhanta models eclipses based on:

- A) Retrograde motion
- B) Angular distance and shadow projection
- C) Planetary conjunctions
- D) Sidereal time
- ☒ **Correct Answer: B**

16. In Aryabhata's model, planetary motion is explained using:

- A) Epicycles
- B) Retrograde loops
- C) Ellipses
- D) Spiral orbits
- ☒ **Correct Answer: A**

17. The concept of retrograde motion is addressed in:

- A) Spherical trigonometry
- B) Planetary mean positions
- C) Observational astronomy
- D) Apparent vs true planetary positions
- ☒ **Correct Answer: D**

18. Planetary motion was calculated using:

- A) Trigonometric tables
- B) Star alignments
- C) Solar flares
- D) Magnetic fields
- ☒ **Correct Answer: A**

19. The use of **epicycles** in planetary motion was shared with:

- A) Newtonian models
- B) Greek astronomy (Ptolemy)
- C) Kepler's laws
- D) Copernican system
- ☒ **Correct Answer: B**

20. Which of these planets' motion is most challenging to model without modern instruments?

- A) Mars
- B) Jupiter
- C) Saturn
- D) Mercury
- ☒ **Correct Answer: D**

21. Aryabhata emphasized the Sun's role in:

- A) Lunar motion
- B) Causing eclipses only
- C) Governing planetary motion
- D) Producing retrograde motion
- ☒ **Correct Answer: C**

22. The planetary positions in Suryasiddhanta are determined relative to:

- A) The Earth's center
- B) The Moon's phases

- C) The Sun's position
- D) The celestial poles

• ☒ **Correct Answer: C**

23. In ancient Indian models, the Sun was considered:

- A) Stationary in the center
- B) Orbiting Earth
- C) A key point for planetary calculations
- D) Just a source of heat

• ☒ **Correct Answer: C**

24. The concept of mean and true position of planets relies heavily on:

- A) Solar calendar
- B) Sun's ecliptic movement
- C) Magnetic field of planets
- D) Star charts

• ☒ **Correct Answer: B**

25. Aryabhata's view of the Sun's effect on planet motion is similar in purpose to:

- A) Newton's law of gravity
- B) Kepler's laws
- C) Ptolemaic epicycles
- D) Einstein's relativity

• ☒ **Correct Answer: B**

26. How many zodiac signs are described in Suryasiddhanta?

- A) 10
- B) 12
- C) 14
- D) 27

• ☒ **Correct Answer: B**

27. The zodiac division in Indian astronomy is based on:

- A) Moon's cycle
- B) Solar motion along ecliptic
- C) Star positions
- D) Planetary conjunctions

• ☒ **Correct Answer: B**

28. Each zodiac sign in Indian astronomy covers how many degrees of the sky?

- A) 20°
- B) 25°
- C) 30°
- D) 15°

✓ **Correct Answer: C**

29. Nakshatras differ from zodiac signs because:

- A) They are imaginary
- B) They are based on Moon's path
- C) They belong to modern astronomy
- D) They were invented by Greeks

✓ **Correct Answer: B**

30. The zodiac system in Aryabhata's astronomy is:

- A) Arbitrary
- B) Based on sun's transition through constellations
- C) Irrelevant
- D) Meant for astrology only

✓ **Correct Answer: B**

31. Aryabhata described the solar system as:

- A) Sun-centric
- B) Earth-centric
- C) Moon-centric
- D) Stationary model

✓ **Correct Answer: B**

32. Which celestial bodies were considered part of the solar system in Suryasiddhanta?

- A) Planets and comets
- B) Sun, Moon, Planets
- C) Only Earth and Moon
- D) Stars

✓ **Correct Answer: B**

33. The ancient Indian model of the solar system assumed:

- A) Circular orbits only
- B) Spiral motion

- C) Linear trajectories
- D) Elliptical orbits
- ☒ **Correct Answer: A**

34. Which of the following is **not** a graha (planet) in classical Indian astronomy?

- A) Mercury
- B) Jupiter
- C) Neptune
- D) Venus
- ☒ **Correct Answer: C**

35. The ordering of planets in Suryasiddhanta was based on:

- A) Mass
- B) Speed and distance
- C) Brightness
- D) Color
- ☒ **Correct Answer: B**

36. The speed of planets in Aryabhata's system is:

- A) Constant for all planets
- B) Inversely proportional to their distance
- C) Based on epicycles
- D) Random
- ☒ **Correct Answer: B**

37. Which planet was observed to have the **fastest** movement across the sky?

- A) Mars
- B) Saturn
- C) Mercury
- D) Jupiter
- ☒ **Correct Answer: C**

38. Retrograde motion of planets is explained in terms of:

- A) Epicycle speeds
- B) Actual backward movement
- C) Sun's gravitational field
- D) Earth's tilt
- ☒ **Correct Answer: A**

39. The relative speed of planets is measured against:

- A) Moon's motion
- B) Fixed stars
- C) Solar calendar
- D) Earth's rotation
- ☒ **Correct Answer: B**

40. The difference in speed between inner and outer planets is due to:

- A) Distance from Earth
- B) Sun's influence and orbital paths
- C) Atmospheric drag
- D) Astrological effects
- ☒ **Correct Answer: B**

41. Aryabhata calculated distance to the Moon in terms of:

- A) Yojanas
- B) Kilometers
- C) Angulas
- D) Kalas
- ☒ **Correct Answer: A**

42. The ancient value for distance to the Moon was approximately:

- A) 100,000 yojanas
- B) 50,000 yojanas
- C) 500 yojanas
- D) 500,000 yojanas
- ☒ **Correct Answer: A**

43. The distances in Suryasiddhanta were mostly:

- A) Conceptual
- B) Accurate to modern standards
- C) Based on ratios and angular observations
- D) Measured using satellites
- ☒ **Correct Answer: C**

44. Latitude in Indian astronomy refers to:

- A) Planet's position above/below ecliptic
- B) Distance from Earth

🌙 4.2.1 On Lunar Phases (Sutra – Verse 2.10)

45. According to Varahamihira, the phases of the Moon occur due to:

- A) The Moon's own light
- B) The shadow of other planets
- C) The reflection of sunlight on the Moon
- D) The alignment of zodiac signs
- ☒ **Correct Answer: C**

46. The visibility of the Moon's phases was explained by Varahamihira using:

- a. A) Spiritual symbolism
- b. B) Astrological beliefs
- c. C) Geometric angles between Earth, Sun, and Moon
- d. D) Wind directions
- e. ☒ **Correct Answer: C**

47. Which lunar phase marks the start of the **waxing** cycle?

- a. A) Full Moon
- b. B) New Moon
- c. C) Third Quarter
- d. D) First Quarter
- e. ☒ **Correct Answer: B**

🌟 4.2.2 On the Movements of the Stars (Sutra – Verse 2.18)

48. Varahamihira recognized the apparent movement of stars as:

- a. A) Stationary phenomenon
- b. B) Illusion caused by Sun
- c. C) Result of Earth's rotation
- d. D) Influence of Rahu and Ketu
- e. ☒ **Correct Answer: C**

49. The Nakshatras in Varahamihira's system are:

- a. A) Planetary deities
- b. B) Lunar mansions along the ecliptic
- c. C) Astrological houses
- d. D) Western constellations
- e. ☒ **Correct Answer: B**

50. How many Nakshatras are described in Indian astronomy, as per Varahamihira?

- a. A) 12
- b. B) 24
- c. C) 27
- d. D) 30
- e. ☒ **Correct Answer: C**

4.2.3 Ecliptic Latitude and Longitude

51. Ecliptic latitude refers to a planet's:

- a. A) Angular distance from Earth's equator
- b. B) North-south distance from ecliptic
- c. C) Distance from Moon
- d. D) Height in the sky
- e. ☒ **Correct Answer: B**

52. Ecliptic longitude is measured:

- a. A) From the equator
- b. B) From the Moon's position
- c. C) Along the ecliptic from the vernal equinox
- d. D) Vertically from horizon
- e. ☒ **Correct Answer: C**

53. The need for both latitude and longitude in planetary models helps:

- a. A) Predict brightness of stars
- b. B) Simplify timekeeping
- c. C) Accurately locate celestial objects
- d. D) Define astrological houses
- e. ☒ **Correct Answer: C**

4.2.4 Sidereal and Tropical Years

54. A **sidereal year** is the time taken for the Earth to:

- A) Return to same zodiac sign
- B) Orbit once around the Sun relative to fixed stars
- C) Complete a lunar cycle
- D) Align with Mars
- ☒ **Correct Answer: B**

55. A **tropical year** is measured by:

- A) Moon's orbit around Earth
- B) Sun's return to the vernal equinox
- C) Fixed star alignment
- D) Rahu-Ketu axis rotation
- ☒ **Correct Answer: B**

56. The difference between sidereal and tropical years is due to:

- A) Sunspot cycles
- B) Precession of equinoxes
- C) Lunar perturbation
- D) Atmospheric drag
- ☒ **Correct Answer: B**

4.2.5 Planetary Conjunctions and Aspects

57. A planetary **conjunction** occurs when:

- A) Planets rise at same time
- B) Two or more planets appear close together in the sky
- C) Planets are on opposite sides of Earth
- D) Eclipse happens
- ☒ **Correct Answer: B**

58. In Indian astronomy, **aspects** between planets are based on:

- A) Brightness difference
- B) Planetary sizes
- C) Angular distances and influence
- D) Magnetic field interactions
- ☒ **Correct Answer: C**

59. Varahamihira's study of conjunctions aimed to:

- A) Track planetary origin
- B) Understand zodiac creation
- C) Predict planetary visibility and influence
- D) Measure solar flares
- ☒ **Correct Answer: C**

SHORT QUESTION / ANSWERS :

4.1 Ancient Indian Astronomy from Suryasiddhanta by Aryabhata

4.1.1 Motion of the Earth

1. **What does Aryabhata say about Earth's movement?**

Aryabhata proposed that the Earth rotates on its axis, which explains the apparent daily movement of the stars from east to west.

2. **How did Aryabhata explain the illusion of moving stars?**

He used the analogy of a person in a moving boat seeing the shoreline move backward, explaining the apparent movement of stars.

3. **How was Aryabhata's view different from traditional beliefs?**

Unlike the traditional geocentric view, Aryabhata suggested that Earth rotates, which was revolutionary for his time.

4. **What effect does Earth's rotation have in Aryabhata's model?**

Earth's rotation causes the apparent rising and setting of celestial bodies, eliminating the need to assume the sky itself rotates.

4.1.2 Length of the Year

5. **How long is a year according to Aryabhata?**

Aryabhata calculated the sidereal year as 365.258 days, which is very close to the modern value.

6. **What is the basis of Aryabhata's year calculation?**

He used astronomical observations and mathematical calculations based on planetary and solar positions.

7. **How accurate was Aryabhata's calculation of the year?**

His value differed from the modern value by only a few minutes, showing remarkable precision.

8. **What did Aryabhata's work contribute to calendar making?**

It provided accurate year lengths and motion data, aiding in designing solar and lunar calendars.

4.1.3 Lunar and Solar Eclipses

9. **How did Aryabhata explain solar eclipses?**

He stated that solar eclipses occur when the Moon passes between the Earth and the Sun, blocking sunlight.

10. **What causes lunar eclipses according to Suryasiddhanta?**

Lunar eclipses occur when the Earth's shadow falls on the Moon during full moon phases.

11. **Did Aryabhata believe in mythological causes of eclipses?**

No, he rejected myths and provided a scientific explanation based on celestial alignment and shadows.

12. **Why was Aryabhata's eclipse theory important?**

It demonstrated early use of scientific reasoning and rejected superstition, paving the way for observational astronomy.

4.1.4 The Motion of Planets

13. **What system did Aryabhata use to explain planetary motion?**

He used the geocentric model with epicycles and deferents to explain apparent planetary paths.

14. **What is retrograde motion, and how is it explained?**

Retrograde motion is the apparent backward movement of planets; Aryabhata explained it using epicycles.

15. Why was planetary motion complex to model?

Because of varying speeds and retrograde motion, accurate modeling required detailed mathematical tools.

16. Which planets were primarily studied for motion?

Planets visible to the naked eye—Mercury, Venus, Mars, Jupiter, and Saturn—were modeled in detail.

 4.1.5 Influence of the Sun on Planetary Motion

17. What is the Sun's role in planetary motion in Aryabhata's model?

The Sun's position influenced planetary motion calculations and was central to determining their orbits.

18. How does the Sun affect the visibility of planets?

The position of the Sun affects when and where planets are visible in the sky during conjunctions or oppositions.

19. What did Aryabhata understand about the Sun's influence?

He recognized the Sun as a central reference for determining the apparent motions and conjunctions of planets.

20. Why was solar influence important in ancient astronomy?

It allowed for better prediction of planetary positions and celestial events like eclipses and retrograde motion.

 4.1.6 Zodiac and Signs

21. What are the zodiac signs in Indian astronomy?

There are 12 zodiac signs (rāśis), each covering 30° of the celestial sphere along the ecliptic.

22. How do zodiac signs relate to planetary motion?

They mark divisions of the sky through which planets and the Sun appear to move, aiding in timing and predictions.

23. What is the starting point of zodiac measurement?

It begins with Aries (Mesha), starting from the vernal equinox, as per ancient Indian astronomy.

24. What role do nakshatras play in relation to zodiac signs?

Nakshatras are subdivisions of the zodiac, with 27 or 28 lunar constellations used primarily in lunar calculations.

 4.1.7 Solar System

25. How did Aryabhata describe the solar system?

He placed the Earth at the center (geocentric), with celestial bodies including the Sun and planets orbiting around.

26. Which celestial bodies are considered grahas?

Grahas include the five visible planets, the Sun, Moon, and shadow planets Rahu and Ketu.

27. What role does Earth play in Aryabhata's solar system?

Earth is stationary in the center, and all observable motions are relative to it.

28. How is the solar system viewed mathematically?

Aryabhata used calculations and geometric models to describe planetary positions and timings.

 4.1.8 Speed of Planets

29. How did Aryabhata calculate the speed of planets?

He used angular motion and revolutions per yuga (cosmic cycle) to estimate the speed of each planet.

30. Which planet has the fastest movement in the sky?

Mercury, being closest to the Sun, appears to move fastest relative to background stars.

31. How does the distance from Earth affect a planet's speed?

Generally, the farther the planet, the slower it appears to move against the stellar background.

32. What mathematical tools did Aryabhata use?

He used trigonometry, sine tables, and positional geometry to calculate speed and motion.

 4.1.9 Planetary Distances from Earth to Moon

33. How did Aryabhata measure distance to the Moon?

He used a unit called yojana and estimated the Moon's distance based on angular size and motion.

34. What was the estimated distance to the Moon?

The Moon was placed at a distance of about 324,000 yojanas from Earth in ancient texts.

35. Why are distance calculations significant?

They allowed astronomers to estimate orbital periods and eclipse timings accurately.

36. What method was used for these measurements?

Observational geometry, using eclipse data and apparent diameters, helped derive planetary distances.

 4.1.10 Latitude and Longitude of Planets

37. What is planetary latitude in ancient astronomy?

It refers to the angular distance of a planet north or south of the ecliptic.

38. What is planetary longitude?

Longitude is the angular distance measured along the ecliptic from a fixed point, usually the vernal equinox.

39. Why are latitude and longitude important in astronomy?

They help in precisely locating planets and stars in the celestial sphere for predictions and charts.

40. How were these values used in calculations?

They were crucial in determining planetary conjunctions, retrograde motion, and eclipse possibilities.